

Assignment 3 | Bridge Pattern

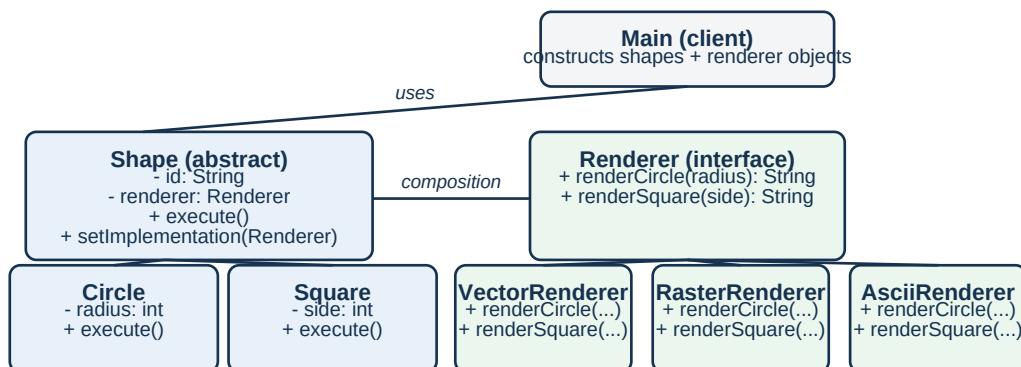
Shape Drawing Renderer · Topic A · Astana IT University · ShP-2216 · 2026–2027

Author	Merey Kuatbay
Program	6B06102 Software Engineering
Source commit	276269d14e29dc35d963890acc68e2d9ac83ce46
Base commit	30798a7b83e3050f9795afaf0ecacc3d05fd74b4

Design and rationale

The abstraction dimension is the shape to draw: **Circle** retains a radius and **Square** retains a side length. The implementation dimension is the rendering format: **VectorRenderer**, **RasterRenderer**, and the extension **AsciiRenderer**. The abstract **Shape** owns an interface-typed **Renderer** reference supplied by its constructor. Each shape delegates to that reference, so either hierarchy can vary without creating a separate subclass for every pair.

UML class diagram



Two independently varying hierarchies. Each `execute()` call delegates to the selected `Renderer`. The link is composition/delegation: `Shape` stores `Renderer` and invokes its operation. `Main` is the client that selects a shape and implementation. The same `Circle` instance accepts a new `Renderer` at runtime through `setImplementation(...)`.

Annotated code excerpts

Five concrete practices visible in the submitted source

1. Separate responsibilities — Shape.java

```
protected Renderer renderer;  
public abstract String execute();
```

Shape owns the abstraction-side identity and bridge reference; renderer classes own representation strings. Each side changes for a different reason.

2. Meaningful names — Circle.java

```
private final int radius;  
public int getRadius() { return radius; }
```

The domain-specific name makes the retained measurement explicit and helps the T5 check verify the same value after switching.

3. Small focused methods — AsciiRenderer.java

```
public String renderCircle(int radius) {  
    return "ASCII circle radius=" + radius;  
}
```

This operation renders one shape in one format. Its output is deterministic and easy for the client demo to compare.

4. Avoid duplicated check workflow — Main.java

```
private static void check(String id, String classes,  
    String expected, String actual) {  
    report(id, expected.equals(actual), ...);  
}
```

T1–T4, T6, and T7 share one comparison and reporting path. report(...) computes PASS/FAIL from the comparison and prints expected output only on failure.

5. Encapsulate domain state — Shape.java and Square.java

```
private final String id;  
private final int side;
```

ID and shape dimensions are private and immutable after construction. They are exposed through narrow getters where the demo needs evidence; implementation replacement does not rewrite them.

Extension, trade-off, and references

Independent I3 extension

The initial implementation was committed at 30798a7b83e3050f9795afaf0ecacc3d05fd74b4. The extension adds **AsciiRenderer** and updates **Main** to demonstrate T6 and T7. The existing abstraction classes, **Renderer** interface, **VectorRenderer**, and **RasterRenderer** remain unchanged. `extension.diff` records the source-only diff from the base commit to source commit 276269d14e29dc35d963890acc68e2d9ac83ce46.

Runtime evidence

The included `demo-output.txt` is captured from `java -cp out Main --demo`. It reports all seven checks and a computed **SUMMARY: 7/7 PASS**. T5 uses reference equality (`==`) and verifies the ID and radius remain unchanged while the rendering result changes from **Vector** to **Raster**.

Trade-off and Adapter comparison

Bridge adds an abstraction and an interface, which introduces indirection and more types than a single renderer-specific shape. That cost is useful when the shape and output format are both expected to vary. Bridge separates two designed dimensions; Adapter wraps an existing interface whose shape does not match the one a client expects. The **Renderer** contract here is the planned implementation API, so no legacy or incompatible API is being adapted.

Build and sources

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

References

1. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Structural patterns: Bridge and Adapter. ISBN 0-201-63361-2.
2. Oracle. *Java Language Specification, Java SE 17 Edition*. <https://docs.oracle.com/javase/specs/jls/se17/jls17.pdf>
3. Astana IT University, School of Software Engineering. *Assignment 3 | Bridge Pattern*, course handout, 2026–2027.